```cs
using System.Collections.Generic;
using UnityEngine;

public class Inventory : MonoBehaviour
{
    public int maxSlot = 10;
    private List<string> items = new List<string>();

    public bool AddItem(string itemName)
    {
        if (items.Count < maxSlot)
        {
            items.Add(itemName);
            return true;
        }

        return false;
    }

    public void RemoveItem(string itemName)
    {
        items.Remove(itemName);
    }

    public List<string> GetItems()
    {
        return new List<string>(items);
    }
}=
```

```cs
using UnityEngine;

public enum ElementType
{
    Fire,
    Water,
    Earth
}

public class DamageCalculator : MonoBehaviour
{
    private float GetElementMultiplier(
        ElementType attacker,
        ElementType defender)
    {
        if (attacker == defender)
            return 1.0f;

        bool isSuperEffective =
            (attacker == ElementType.Fire && defender == ElementType.Earth) ||
            (attacker == ElementType.Earth && defender == ElementType.Water) ||
            (attacker == ElementType.Water && defender == ElementType.Fire);

        return isSuperEffective ? 1.5f : 0.5f;
    }

    public int CalculateDamage(
        int baseDamage,
        ElementType attackerType,
        ElementType defenderType,
        int defenderDefense)
    {
        float damage = baseDamage;

        damage *= GetElementMultiplier(
            attackerType,
            defenderType
        );

        bool isCritical = Random.Range(0f, 1f) < 0.2f;

        if (isCritical)
        {
            damage *= 2f;
        }

        damage -= defenderDefense;

        int finalDamage = Mathf.Max(
            1,
            Mathf.RoundToInt(damage)
        );
```



```cs
}
```

```cs
using UnityEngine;

public abstract class GameCharacter : MonoBehaviour
{
    public string namaKarakter;
    public int hpMaksimum;
    protected int hpSaatIni;

    protected virtual void Awake()
    {
        hpSaatIni = hpMaksimum;
    }

    public void TakeDamage(int dmg)
    {
        hpSaatIni -= dmg;

        if (hpSaatIni < 0)
            hpSaatIni = 0;
    }

    public bool IsAlive()
    {
        return hpSaatIni > 0;
    }

    public abstract void Attack();
}
```

```cs
using UnityEngine;

public class Player : GameCharacter
{
    public int exp = 0;
    public int level = 1;

    public override void Attack()
    {
        Debug.Log(namaKarakter + " menyerang menggunakan senjata!");
    }

    public void GainExp(int jumlahExp)
    {
        exp += jumlahExp;

        if (exp >= level * 100)
        {
            LevelUp();
        }
    }

    private void LevelUp()
    {
        level++;
        hpMaksimum += 20;

        Debug.Log(
            namaKarakter +
            " naik ke Level " +
            level +
            "!"
        );
    }
}
```

```cs
using System.Collections.Generic;
using UnityEngine;

public class EnemySpawner : MonoBehaviour
{
    private GameObject player;
    private Rigidbody playerRb;

    private List<EnemyAI> enemyAIList =
        new List<EnemyAI>();

    void Start()
    {
        player = GameObject.Find("Player");

        playerRb = player.GetComponent<Rigidbody>();

        GameObject[] enemies =
            GameObject.FindGameObjectsWithTag("Enemy");

        foreach (GameObject enemy in enemies)
        {
            enemyAIList.Add(
                enemy.GetComponent<EnemyAI>()
            );
        }
    }

    void Update()
    {
        foreach (EnemyAI enemyAI in enemyAIList)
        {
            if (enemyAI == null)
                continue;

            float distance = Vector3.Distance(
                enemyAI.transform.position,
                player.transform.position
            );

            if (distance < 5f)
            {
                enemyAI.ChasePlayer(playerRb);
            }
        }
    }
}
```

```cs
using System.Collections.Generic;
using UnityEngine;

public class SpatialHashGrid
{
    private readonly float cellSize;

    private readonly Dictionary<(int, int), List<GameObject>> cells =
        new Dictionary<(int, int), List<GameObject>>();

    public SpatialHashGrid(float cellSize)
    {
        this.cellSize = cellSize;
    }

    private (int, int) GetCellCoord(Vector3 pos)
    {
        int cx = Mathf.FloorToInt(pos.x / cellSize);
        int cy = Mathf.FloorToInt(pos.z / cellSize);

        return (cx, cy);
    }

    public void Clear()
    {
        cells.Clear();
    }

    public void Insert(GameObject obj)
    {
        var coord = GetCellCoord(
            obj.transform.position
        );

        if (!cells.TryGetValue(
            coord,
            out var list))
        {
            list = new List<GameObject>();
            cells[coord] = list;
        }

        list.Add(obj);
    }

    public List<GameObject> GetNearbyObjects(
        GameObject obj)
    {
        var result =
            new List<GameObject>();

        var (cx, cy) =
            GetCellCoord(
                obj.transform.position
            );

        for (int dx = -1; dx <= 1; dx++)
        {
            for (int dy = -1; dy <= 1; dy++)
            {
                if (cells.TryGetValue(
                    (cx + dx, cy + dy),
                    out var list))
                {
                    result.AddRange(list);
                }
            }
        }

        return result;
    }
}
```

```cs
using System.Threading.Tasks;
using UnityEngine;

public class PlayerData
{
    public int gold;
    public int level;
}

public class SaveSystem : MonoBehaviour
{
    private PlayerData data =
        new PlayerData
        {
            gold = 0,
            level = 1
        };

    public void AddGold(int amount)
    {
        data.gold += amount;
    }

    public void SaveToFileAsync()
    {
        PlayerData snapshot =
            new PlayerData
            {
                gold = data.gold,
                level = data.level
            };

        Task.Run(() =>
        {
            string json =
                JsonUtility.ToJson(snapshot);

            System.IO.File.WriteAllText(
                "save.json",
                json
            );
        });
    }
}
```

```cs
using System.Collections.Generic;
using UnityEngine;

public class ObjectPool<T> where T : Component
{
    private readonly Queue<T> pool =
        new Queue<T>();

    private readonly T prefab;
    private readonly Transform parent;

    public ObjectPool(
        T prefab,
        int prewarmCount,
        Transform parent = null)
    {
        this.prefab = prefab;
        this.parent = parent;

        for (int i = 0; i < prewarmCount; i++)
        {
            T obj =
                Object.Instantiate(
                    prefab,
                    parent
                );

            obj.gameObject.SetActive(false);

            pool.Enqueue(obj);
        }
    }

    public T Get()
    {
        T obj =
            pool.Count > 0
            ? pool.Dequeue()
            : Object.Instantiate(
                prefab,
                parent
            );

        obj.gameObject.SetActive(true);

        (obj as IPoolable)?
            .OnSpawnFromPool();

        return obj;
    }

    public void Release(T obj)
    {
        (obj as IPoolable)?
            .OnReturnToPool();

        obj.gameObject.SetActive(false);

        pool.Enqueue(obj);
    }
}

public interface IPoolable
{
    void OnSpawnFromPool();
    void OnReturnToPool();
}
```